

**Department of Electrical Engineering
The Superior University, Lahore**

Computer Programming

**Experiment No.10
Introduction to Functions**

Prepared for:

EISHA NAWAZ

By:

Name: Muneba Rehman

ID: SU91-BSEEM-F25-010

Section: A

Semester: 2nd

Total Marks: _____10_____

Obtained Marks: _____

Signature: _____

Date: _____

Experiment No.10
Introduction to Function
(Rubrics)

Name: _____

ID: _____

A. PSYCHOMOTOR

Serial No.	Criteria	Allocated Marks	Unacceptable 0%	Poor 25%	Fair 50%	Good 75%	Excellent 100%	Total Obtained
1	Syntax handling	1	0	0.25	0.5	0.75	1	
2	Program Coding Skills	2	0	0.5	1	1.5	2	
3	Accuracy in Output Results	3	0	0.75	1.5	2.25	3	
Sub-Total		6	Sub Total Marks Obtained in Psychomotor (P)					

B. AFFECTIVE

Serial No.	Criteria	Allocated Marks	Unacceptable 0%	Poor 25%	Fair 50%	Good 75%	Excellent 100%	Total Obtained
1	Assigned Tasks	2	0	0.5	1	1.5	2	
2	Lab Report	2	0	0.5	1	1.5	2	
Sub-Total		4	Sub Total Marks Obtained in Affective (A)					

Instructor Name: EISHA NAWAZTotal Marks (P+A): 10

Instructor Signature: _____

Obtained Marks (P+A): _____

CONTENTS OF LAB 10

1	OBJECTIVES	4
2	FUNCTIONS IN C++	4
3	WHY DO WE USE FUNCTIONS?	5
4	DEFINING A FUNCTION	5
5	VOID FUNCTIONS (NO RETURN VALUE)	5
6	FUNCTIONS WITH PARAMETERS	6
7	FUNCTIONS THAT RETURN A VALUE	7
8	FUNCTION DECLARATIONS	8
9	TYPES OF FUNCTION	9
9.1	C++ USER DEFINED FUNCTIONS	9
9.2	FUNCTION DECLARATION	10
9.3	CALLING A FUNCTION	10
9.3.1	Example : Displaying A Text	11
9.4	FUNCTION PARAMETERS	11
9.4.1	Example: Function with Parameters	12
9.5	RETURN STATEMENT	14
9.5.1	Example: Add Two Numbers	14
9.6	BENEFITS OF USER-DEFINED FUNCTIONS	16
10	9. VOID VS VALUE-RETURNING FUNCTIONS — SIDE BY SIDE	16
11	10. COMMON MISTAKES TO AVOID	16
12	11. SOLVED PRACTICE PROBLEM	17
13	QUICK RECAP	18
14	Lab Tasks	18
14.1.1	(a) Void Function — No Parameters	Error! Bookmark not defined.
14.1.2	(b) Void Function — With Parameters	Error! Bookmark not defined.
14.1.3	(c) Value-Returning Function	Error! Bookmark not defined.
14.1.4	(d) Function Prototype	Error! Bookmark not defined.

15	Home Task	18
15.1.1	(a) Value-Returning Function	Error! Bookmark not defined.
15.1.2	(b) Library Function	Error! Bookmark not defined.
15.1.3	(c) Value-Returning Function — Decision Making	Error! Bookmark not defined.

Introduction to Functions

1 OBJECTIVES

- Understand what a function is and why programs are broken into functions.
- Learn the anatomy of a function: return type, name, parameters, and body.
- Practice writing void functions and functions that return a value.
- Learn how and why function prototypes are used.
- Use built-in (library) functions such as `sqrt()` and `pow()`.

2 FUNCTIONS IN C++

A function is a named, reusable block of code that performs one specific task. Think of it like a kitchen appliance: a blender always blends whatever you put in it, you do not rebuild it every time you want a smoothie. In the same way, once you write a function, you can “call” it as many times as you like instead of retyping the same code.

Every C++ program already uses at least one function: **main()**. Everything you have written so far has lived inside **main()**. Now you will learn to create your own additional functions and have `main()` call them.

i Tip: Breaking a big problem into small functions is called “divide and conquer.” Each function solves one small, clear piece of the overall problem.

A function **declaration** tells the compiler about a function's name, return type, and parameters. A function **definition** provides the actual body of the function.

The C++ standard library provides numerous built-in functions that your program can call.

A function is known with various names like a method or a sub-routine or a procedure etc.

3 WHY DO WE USE FUNCTIONS?

- Reusability: write the code once, call it as many times as needed.
- Readability: main() becomes a short, clear summary of what the program does.
- Easier debugging: if something goes wrong, you know exactly which small function to check.
- Teamwork: different people can write different functions for the same program.

4 DEFINING A FUNCTION

The general form of a C++ function definition is as follows –

```
return_type function_name( parameter_list )
{
    // body: the statements that do the work
    // (optional) return a value here
}
```

A C++ function definition consists of a function header and a function body. Here are all the parts of a function –

- **Return Type** – A function may return a value. The **return_type** is the data type of the value the function returns. Some functions perform the desired operations without returning a value. In this case, the return_type is the keyword **void**.
- **Function Name** – This is the actual name of the function. The function name and the parameter list together constitute the function signature.
- **Parameters** – A parameter is like a placeholder. When a function is invoked, you pass a value to the parameter. This value is referred to as actual parameter or argument. The parameter list refers to the type, order, and number of the parameters of a function. Parameters are optional; that is, a function may contain no parameters.
- **Function Body** – The function body contains a collection of statements that define what the function does.

5 VOID FUNCTIONS (NO RETURN VALUE)

A void function performs an action (like printing something) but does not hand any value back to whoever called it.

```
#include <iostream>
using namespace std;

// function declaration + definition
void greet()
{
    cout << "Hello there!";
}

int main()
{
    greet();    // calling the function
    return 0;
}
```

Output

Hello there!

i Tip: Notice greet() has empty parentheses — it takes no parameters — and its return type is void, so it never uses a return statement to send back a value.

6 FUNCTIONS WITH PARAMETERS

A parameter is a placeholder that receives a value when the function is called. This lets the same function behave differently depending on what you pass in.

```
#include <iostream>
using namespace std;

void displayNum(int n1, float n2)
{
```

```
    cout << "The int number is " << n1 << endl;
    cout << "The float number is " << n2 << endl;
}

int main()
{
    int num1 = 5;
    float num2 = 5.5;
    displayNum(num1, num2);    // num1, num2 are passed as arguments
    return 0;
}
```

Output

The int number is 5

The float number is 5.5

i Tip: The values num1 and num2 in main() are called arguments. Inside the function they are received into the parameters n1 and n2. The names do not have to match — only the order and type matter.

Important: C++ uses **call by value** by default: the function receives a copy of the argument. Changing a parameter inside the function does not change the original variable in main().

7 FUNCTIONS THAT RETURN A VALUE

Instead of void, a function can be given a real return type (like int or float). Inside the function body, a return statement sends a value back to the caller, and execution of the function stops right there.

```
#include <iostream>
using namespace std;

int add(int a, int b)
{
```

```
    return (a + b);
}

int main()
{
    int sum;
    sum = add(100, 78);    // the returned value is stored in sum
    cout << "100 + 78 = " << sum << endl;
    return 0;
}
```

Output

100 + 78 = 178

i Tip: Because `add()` returns an `int`, the variable that stores its result (`sum`) must also be an `int`. Whatever comes right after `return` is sent back, and any code written after that `return` statement inside the same function never runs.

8 FUNCTION DECLARATIONS

C++ normally needs to see a function before it is called. A function Declaration is a one-line preview return type, name, and parameter types only, ending with a semicolon, placed above `main()`. It lets you define the full function later, even after `main()`.

```
#include <iostream>
using namespace std;

int add(int, int);    // function prototype

int main()
{
    int sum = add(100, 78);    // used before it is fully defined below
}
```



```
    cout << "100 + 78 = " << sum << endl;
    return 0;
}

int add(int a, int b)    // full definition, written after main()
{
    return (a + b);
}
```

Output

100 + 78 = 178

i Tip: A prototype only needs the parameter types, not their names — `int add(int, int);` is perfectly valid. Always end a prototype with a semicolon.

9 TYPES OF FUNCTION

Suppose we need to create a program to create a circle and color it. We can create two functions to solve this problem:

- a function to draw the circle
- a function to color the circle

Dividing a complex problem into smaller chunks makes our program easy to understand and reusable.

There are two types of function:

1. **Standard Library Functions:** Predefined in C++
2. **User-defined Function:** Created by users.

9.1 C++ USER DEFINED FUNCTIONS

C++ allows the programmer to define their own function.

A user-defined function groups code to perform a specific task and that group of code is given a name (identifier).

When the function is invoked from any part of the program, it all executes the codes defined in the body of the function.

9.2 FUNCTION DECLARATION

The syntax to declare a function is:

```
returnType functionName (parameter1, parameter2,...) {  
    // function body  
}
```

Here's an example of a function declaration.

```
// function declaration  
void greet() {  
    cout << "Hello World";  
}
```

Here,

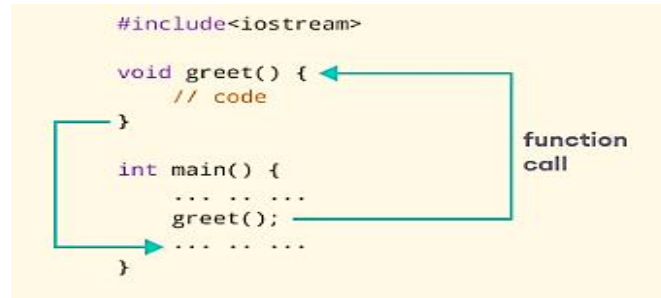
1. the name of the function is greet()
2. the return type of the function is void
3. the empty parentheses mean it doesn't have any parameters
4. the function body is written inside {}

9.3 CALLING A FUNCTION

In the above program, we have declared a function named greet(). To use the greet() function, we need to call it.

Here's how we can call the above greet() function.

```
int main() {  
    // calling a function  
    greet();  
}
```



9.3.1 Example : Displaying A Text

Here, we have not mentioned the size of the array. In such cases, the compiler automatically computes the size.

Code	Output
<pre> #include <iostream> using namespace std; // declaring a function void greet() { cout << "Hello there!"; } int main() { // calling the function greet(); return 0; } </pre>	Hello there!

9.4 FUNCTION PARAMETERS

In C++, if an array has a size n , we can store up to n number of elements in the array. However, what will happen if we store less than n number of elements.

As mentioned above, a function can be declared with parameters (arguments). A parameter is a value that is passed when declaring a function.

For example, let us consider the function below:

```
void printNum(int num) {  
    cout << num;  
}
```

Here, the int variable num is the function parameter.

We pass a value to the function parameter while calling the function.

```
int main() {  
    int n = 7;  
    // calling the function  
    // n is passed to the function as argument  
    printNum(n);  
    return 0;  
}
```

9.4.1 Example: Function with Parameters

Code	Output
<pre>// program to print a text #include <iostream> using namespace std; // display a number void displayNum(int n1, float n2) { cout << "The int number is " << n1; cout << "The double number is " << n2; }</pre>	<pre>The int number is 5 The double number is 5.5</pre>

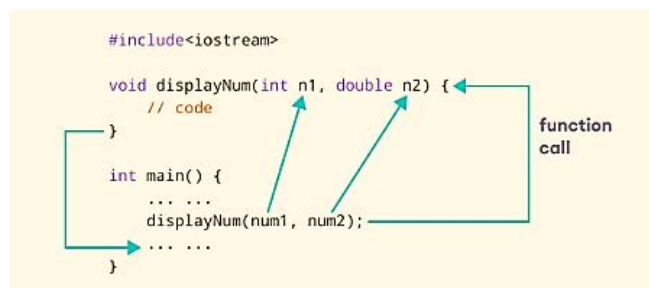
```

int main() {
    int num1 = 5;
    double num2 = 5.5;
    // calling the function
    displayNum(num1, num2);
    return 0;
}

```

In the above program, we have used a function that has one int parameter and one double parameter.

We then pass num1 and num2 as arguments. These values are stored by the function parameters n1 and n2 respectively.



Note: The type of the arguments passed while calling the function must match with the corresponding parameters defined in the function declaration.

Sr. No.	Call Type and Description
1	Call by Value This method copies the actual value of an argument into the formal parameter of the function. In this case, changes made to the parameter inside the function have no effect on the argument.
2	Call by Pointer This method copies the address of an argument into the formal parameter. Inside the function, the address is used to access the actual argument used in the call. This means that changes made to the parameter affect the argument.
3	Call by Reference This method copies the reference of an argument into the formal parameter. Inside the function, the reference is used to access the actual argument used in the call. This means that changes

	made to the parameter affect the argument.
--	--

By default, C++ uses **call by value** to pass arguments. In general, this means that code within a function cannot alter the arguments used to call the function and above mentioned example while calling max() function used the same method.

9.5 RETURN STATEMENT

The example of inserting and printing elements in an array is as follows:

In the above programs, we have used void in the function declaration. For example,

```
void displayNumber() {
    // code
}
```

This means the function is not returning any value.

It's also possible to return a value from a function. For this, we need to specify the returnType of the function during function declaration.

Then, the return statement can be used to return a value from a function.

For example,

```
int add (int a, int b) {
    return (a + b);
}
```

Here, we have the data type int instead of void. This means that the function returns an int value.

The code return (a + b); returns the sum of the two parameters as the function value.

The return statement denotes that the function has ended. Any code after return inside the function is not executed.

9.5.1 Example: Add Two Numbers

The code and output is given as follows:

Code	Output
// program to add two numbers using a function #include <iostream>	100 + 78 = 178

```
using namespace std;

// declaring a function
int add(int a, int b) {
    return (a + b);
}

int main() {
    int sum;

    // calling the function and storing
    // the returned value in sum
    sum = add(100, 78);

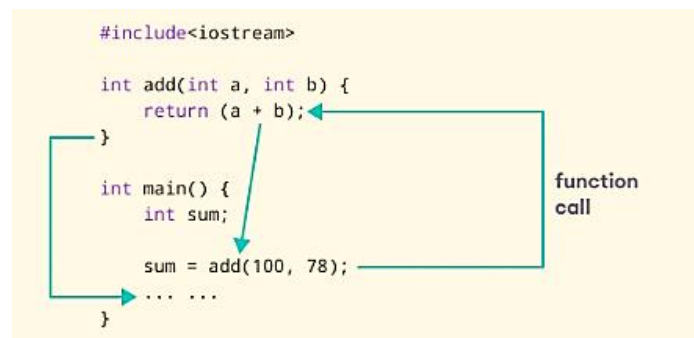
    cout << "100 + 78 = " << sum << endl;

    return 0;
}
```

In the above program, the add() function is used to find the sum of two numbers.

We pass two int literals 100 and 78 while calling the function.

We store the returned value of the function in the variable sum, and then we print it.



Notice that sum is a variable of int type. This is because the return value of add() is of int type.

```
...);
```

9.6 BENEFITS OF USER-DEFINED FUNCTIONS

1. Functions make the code reusable. We can declare them once and use them multiple times.
2. Functions make the program easier as each small task is divided into a function.
3. Functions increase readability.

C++ ships with many ready-made functions so you never have to write them yourself. Math functions like `sqrt()` (square root) and `pow()` (power) live in the `<cmath>` header, so you must `#include <cmath>` to use them.

```
#include <iostream>
#include <cmath>
using namespace std;

int main()
{
    double number = 25.0;
    double squareRoot = sqrt(number);    // library function
    cout << "Square root of " << number << " = " << squareRoot;
    return 0;
}
```

Output

Square root of 25 = 5

10 9. VOID VS VALUE-RETURNING FUNCTIONS — SIDE BY SIDE

	Void Function	Value-Returning Function
Return type	void	int, float, double, etc.
Sends a value back?	No	Yes, using return
Typical use	Printing / displaying something	Calculating a result to use later
Can you store the call in a variable?	No — there is nothing to store	Yes — e.g. <code>int s = add(2,3);</code>

11 10. COMMON MISTAKES TO AVOID

Common Mistake	Why / Fix
----------------	-----------

Forgetting the function prototype when the function is defined after main().	Add a one-line prototype (return type, name, parameter types, semicolon) above main().
Using void but still trying to write return sum;.	A void function cannot return a value. Either change the return type or use a plain return; with nothing after it.
Mismatching argument types and parameter types, e.g. passing a float where an int is expected.	Check the function's parameter list and pass matching types, or the value may be truncated unexpectedly.
Expecting a change made to a parameter inside the function to affect the original variable in main().	Remember C++ uses call by value — the function only gets a copy. The original variable in main() is unchanged.
Calling a function before it is declared or defined, with no prototype.	The compiler reads top to bottom. Either move the definition above main(), or add a prototype above main().

12 11. SOLVED PRACTICE PROBLEM

Even or Odd Checker

Problem: Write a function that takes an integer and returns 1 if it is even, and 0 if it is odd.

Thinking it through: a number is even if there is no remainder when divided by 2. The % (modulus) operator gives us that remainder.

```
#include <iostream>
using namespace std;

int isEven(int n)
{
    if (n % 2 == 0)
        return 1;
    else
        return 0;
}

int main()
{
    int number = 14;
    int result = isEven(number);

    if (result == 1)
```

```

        cout << number << " is even." << endl;
    else
        cout << number << " is odd." << endl;

    return 0;
}

```

Output

14 is even.

Dry run:

Step	Expression	Value
1	n = 14	14
2	n % 2	0
3	condition n % 2 == 0	true
4	return value	1
5	result	1 → prints "14 is even."

13 QUICK RECAP

- A function is a named, reusable block of code that performs one task.
- General shape: `return_type function_name(parameters) { body }`.
- void functions perform an action but return nothing; other functions use return to send back a value.
- A function prototype lets you call a function before its full definition appears in the file.
- C++ passes arguments by value by default — the function works with a copy.

Library functions (like `sqrt()`, `pow()`) are ready-made — just `#include` the right header.

14 LAB TASKS

Resistor Network Analyzer

Scenario: In a circuits lab, you often need to combine resistors in series or parallel and report each one clearly. You will build a small analyzer using functions.

14.1.1 (a) Void Function — No Parameters

Write a void function `showLabHeader()` that prints a banner for the circuits lab, then call it from `main()`.

```
#include <iostream>
using namespace std;

// TODO: write the function showLabHeader() here.
// It should print something like:
// ===== Electrical Engineering Circuit Lab =====

int main()
{
    // TODO: call showLabHeader() here
    return 0;
}
```

Expected Output

```
===== Electrical Engineering Circuit Lab =====
```

14.1.2 (b) Void Function — With Parameters

Write void `displayResistor(int id, float resistance)` that prints a line describing one resistor. Call it twice from `main()` using resistor 1 = 100 ohms and resistor 2 = 220 ohms.

```
// TODO: write void displayResistor(int id, float resistance)
// It should print, e.g.: Resistor R1: 100 ohms

int main()
{
    // TODO: call displayResistor(1, 100) and displayResistor(2, 220)
    return 0;
}
```

Expected Output

Resistor R1: 100 ohms

Resistor R2: 220 ohms

14.1.3 (c) Value-Returning Functions

Write two functions: `float seriesResistance(float r1, float r2)` returning $r1 + r2$, and `float parallelResistance(float r1, float r2)` returning $(r1 * r2) / (r1 + r2)$. Use resistor 1 = 100 ohms and resistor 2 = 220 ohms.

```
// TODO: write float seriesResistance(float r1, float r2)
// TODO: write float parallelResistance(float r1, float r2)

int main()
{
    float r1 = 100, r2 = 220;
    // TODO: call both functions and print the results with cout
    return 0;
}
```

Expected Output

Series resistance: 320 ohms

Parallel resistance: 68.75 ohms

14.1.4 (d) Function Prototype

Move the full definition of `parallelResistance()` to after `main()`, and add a function prototype above `main()` so the program still compiles and runs correctly.

```
#include <iostream>
using namespace std;

// TODO: write the prototype for parallelResistance here
```

```

int main()
{
    float r1 = 100, r2 = 220;

    cout << "Parallel resistance: " << parallelResistance(r1, r2) << " ohms" << endl;

    return 0;
}

// TODO: write the full definition of parallelResistance here

```

Expected Output

Parallel resistance: 68.75 ohms

```

1  #include <iostream>
2  using namespace std;
3  void showLabHeader();
4  void displayResistor(int, float);
5  float seriesResistance(float, float);
6  float parallelResistance(float, float);
7  int main() {
8      showLabHeader();
9      displayResistor(1, 100);
10     displayResistor(2, 220);
11
12     float r1 = 100, r2 = 220;
13
14     cout << "Series resistance: " << seriesResistance(r1, r2) << " ohms" << endl;
15     cout << "Parallel resistance: " << parallelResistance(r1, r2) << " ohms" << endl;
16
17     return 0;
18 }
19 void showLabHeader() {
20     cout << "==== Electrical Engineering Curcuit Lab =====" << endl;
21 }
22
23 void displayResistor(int id, float resistance) {
24     cout << "Resistor R" << id << " = " << resistance << " ohms" << endl;
25 }
26
27 float seriesResistance(float r1, float r2) {
28     return r1 + r2;
29 }
30
31 float parallelResistance(float r1, float r2) {
32     return (r1 * r2) / (r1 + r2);
33 }

```

```
C:\Users\Yahoo Net Cafe\OneDrive\Documents\c++.exe
===== Electrical Engineering Curcuit Lab =====
Resistor R1 = 100 ohms
Resistor R2 = 220 ohms
Series resistance: 320 ohms
Parallel resistance: 68.75 ohms

-----
Process exited after 0.1647 seconds with return value 0
Press any key to continue . . .
```

15 HOME TASK

OHM'S LAW & POWER DISSIPATION CALCULATOR

Scenario: Given a circuit's voltage and resistance, calculate the current and the power dissipated as heat.

15.1.1 (a) Value-Returning Function — Ohm's Law

Write float calculateCurrent(float voltage, float resistance) that returns voltage / resistance. Test it with voltage = 12V and resistance = 4 ohms.

```
// TODO: write float calculateCurrent(float voltage, float resistance)

int main()
{
    // TODO: call the function with voltage=12, resistance=4 and print the result
    return 0;
}
```

Expected Output

Current: 3 A

15.1.2 (b) Library Function

Write float calculatePower(float current, float resistance) that returns the power dissipated, using the formula $P = I^2 \times R$. Use the pow() function from <cmath> to calculate I^2 instead of writing current * current.

```
#include <cmath>

// TODO: write float calculatePower(float current, float resistance)
// use pow(current, 2) inside your formula
```

Expected Output (using current = 3A, resistance = 4 ohms)

Power dissipated: 36 watts

15.1.3 (c) Void Function — Combine Everything

Write void displaySummary(float voltage, float current, float power) that neatly prints all three values together, and call it from main() after computing current and power.

Expected Output

Voltage: 12 V | Current: 3 A | Power: 36 W

```
1 #include <iostream>
2 using namespace std;
3 float calculateCurrent(float voltage, float resistance) {
4     return voltage / resistance;
5 }
6
7 float calculatePower(float current, float resistance) {
8     return current * current * resistance;
9 }
10
11 void displaySummary(float voltage, float current, float power) {
12     cout << "Voltage: " << voltage << " V | Current: " << current << " A | Power: " << power << " watts" << endl;
13 }
14
15 int main() {
16
17     cout << "Current: " << calculateCurrent(12, 4) << " A" << endl;
18     cout << "Power dissipated: " << calculatePower(3, 4) << " watts" << endl;
19     float voltage = 12, resistance = 4;
20     float current = voltage / resistance;
21     float power = current * current * resistance;
22
23     displaySummary(voltage, current, power);
24
25     return 0;
26 }
```

C:\Users\Yahoo Net Cafe\OneDrive\Documents\c++.exe

```
Current: 3 A
Power dissipated: 36 watts
Voltage: 12 V | Current: 3 A | Power: 36 watts
-----
Process exited after 0.2841 seconds with return value 0
Press any key to continue . . .
```